

CLICK-A-THON 2026 · INMOBI TRACK

# Automated Root-Cause Analyst

A metric moved. It tells you **which segment** did it — in seconds, with numbers you can recompute yourself.

Jalagaara Gang

Rohan M Rao · Ankith Dinakar · Shashank · Shreyas Bharadhwaj

[clickathon.kangasys.com](https://clickathon.kangasys.com)

## THE PROBLEM

# Alerts tell you **what**. Nobody tells you **why**.

Revenue drops four percent. The dashboard turns red. Then an analyst starts slicing — by country, by device, by OS, by app, by advertiser — comparing each cut against a baseline, by hand, for hours.

### 9M

AD EVENTS

Five weeks of requests, fills, impressions, clicks and revenue.

### 11

DIMENSIONS

Region, country, OS, device, format, category, tier, vertical, campaign, app, advertiser.

### 10<sup>4</sup>+

SEGMENT COMBINATIONS

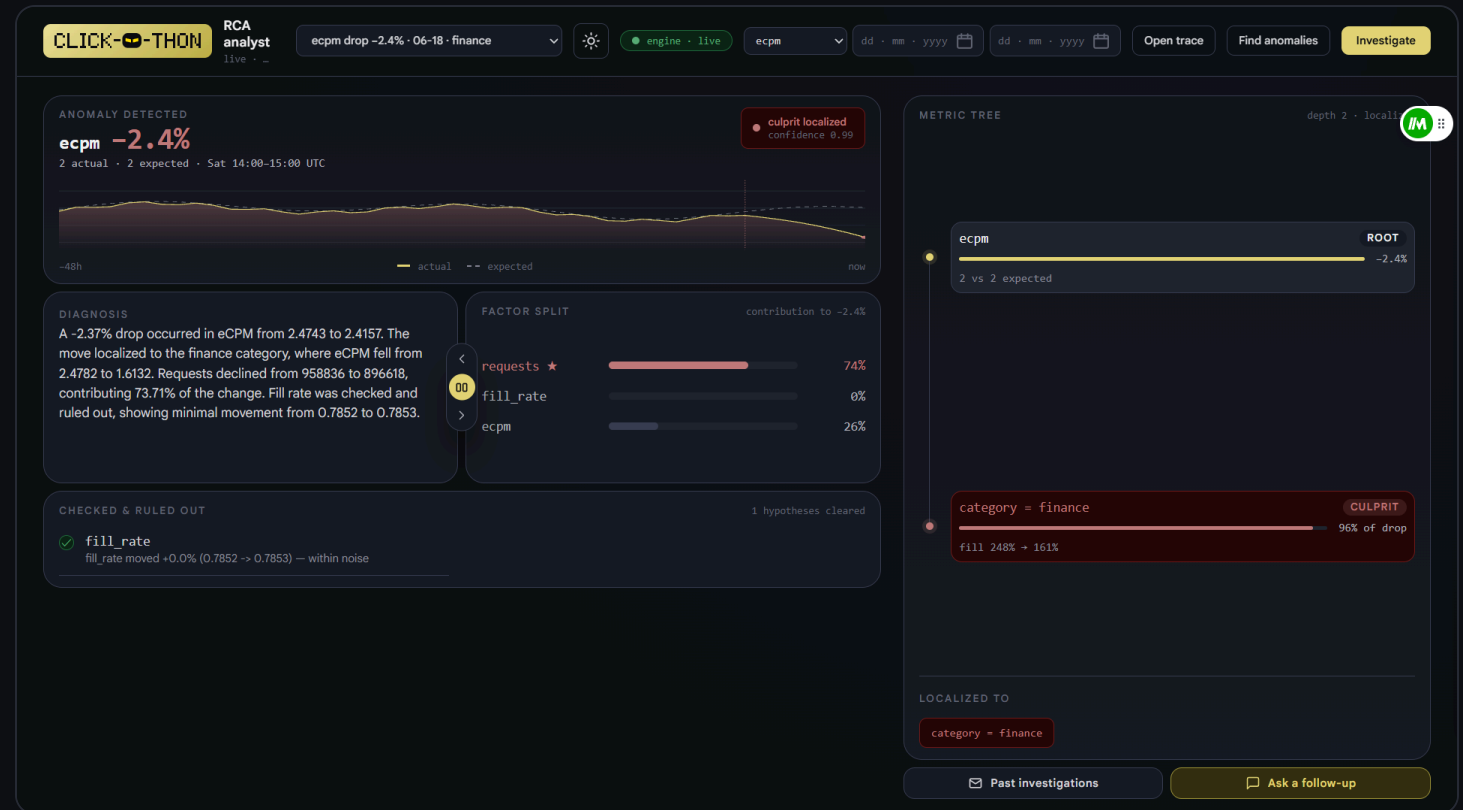
The bottleneck was never the data. It was human attention.

# Ask in plain English. Get the segment, and the evidence.

Three stages, all computed in ClickHouse:

- **Detect** — is this move real, or ordinary noise?
- **Decompose** — revenue = requests × fill rate × eCPM. Which factor moved?
- **Drill down** — which segment inside it is responsible, and what was cleared?

Live on real data, with the diagnosis, the factor split and the ruled-out list all on one screen.



# The obvious way to rank segments is **wrong**.

Rank each segment by how much of the gap it explains, descend into the biggest, repeat. It is the first thing anyone writes — and it fails, for a reason that is not obvious until you measure it.

## What it returns

true cause    **one condition**

what it says **one condition**

AND the busiest tier

AND the busiest region

AND the busiest device

Three extra conditions. Each explains nothing, and each one is wrong.

## Why

For a **uniform** effect, the share of the gap a segment explains is simply **its share of traffic**.

So the ranking does not find the culprit. It finds **the biggest slice** — then the biggest slice inside that, and keeps going until it runs out of dimensions.

# Divide by size. Everything falls out.

Score each segment by a counterfactual — restore its component sums to baseline and measure how much of the gap closes — then divide that by the segment's share of the metric's own volume.

$$\text{lift} = \text{gap this segment closes} \div \text{its share of the volume}$$

≈ 1

MERELY LARGE

Closes about its own share. Big, innocent, and correctly ignored.

>> 1

THE CULPRIT

Closes far more than its size explains. The only kind of segment worth naming.

Descend into the disproportionate one, add it to the filter, recurse. Stop when nothing left is both material and disproportionate.

THE INSIGHT • 3 OF 3

# We implemented both, and measured.

Same engine, same data, same windows — only the ranking rule changed.

RANK BY CONTRIBUTION

**none**

of the known anomalies localized correctly

RANK BY LIFT

**all**

including the hard cases that follow

Both cases are pinned as regression tests — a false positive now fails the build like any other bug.

# Sometimes the right answer is **nobody**.

When a metric drops uniformly across every region and every hour, there is no guilty segment. Naming one is a **false positive** — and a confident, specific, wrong answer is worse than no answer.

The stop criterion catches this by construction: nothing is disproportionate, so the search stops at the population and reports the move without a culprit.

The system also shows its working — every step that led to the verdict, replayable.

The screenshot displays the RCA Assistant interface. At the top, a yellow header bar contains the title "RCA Assistant" and a context ID "Context: 7f1d9043". Below the header, a chat window shows a user message "Hello" and a system response: "Hi! I'm here to help you analyze anomalies and root causes. I see you're looking at an investigation of a significant drop in requests that occurred on June 21, 2026. What would you like to know about it?". A yellow button labeled "Replay The Anomaly" is positioned below the system response. The main content area displays a detailed report titled "Replay: requests drop on Jun 21 00:00 – Jun 22 00:00". The report includes two numbered sections: "1 — Detection" and "2 — Which factor moved". The "Detection" section states that requests came in at 126,052 against an expected 223,079, a move of -43.5% (robust score -28.4). The "Which factor moved" section states that the metric was decomposed into requests (220,775 → 126,052), fill\_rate (0.7846 → 0.7855), and ecpm (2.479 → 2.419), with requests carrying the change. Below the report, a text input field contains "Replay this anomaly end to end..." and a yellow "Send" button. At the bottom of the interface, there are two buttons: "Past investigations" and "Ask a follow-up".

RCA Assistant  
Context: 7f1d9043

Hello

Hi! I'm here to help you analyze anomalies and root causes. I see you're looking at an investigation of a significant drop in requests that occurred on June 21, 2026. What would you like to know about it?

Replay The Anomaly

**Replay: requests drop on Jun 21 00:00 – Jun 22 00:00**

**1 — Detection.** requests came in at **126,052** against an expected **223,079** (same window shifted back 2 weeks (weekday-aligned)), a move of **-43.5%** (robust score -28.4). That cleared both the statistical and the calibrated effect-size gate, so an investigation was opened.

**2 — Which factor moved.** The metric was decomposed (log\_additive) into: requests (220,775 → 126,052); fill\_rate (0.7846 → 0.7855); ecpm (2.479 → 2.419). **requests** carried the change and became the drill-down target.

**4 — Verdict.** No single segment stood out — the move was population-wide, which is

Replay this anomaly end to end... Send

Past investigations Ask a follow-up

# Some causes exist only at an **intersection**.

A segment can look unremarkable on every dimension taken alone, and still be badly broken where two of them meet.

```
scanning one dimension at a time
```

```
os_version          small move  - under the threshold
region              small move  - under the threshold
→ nothing found
```

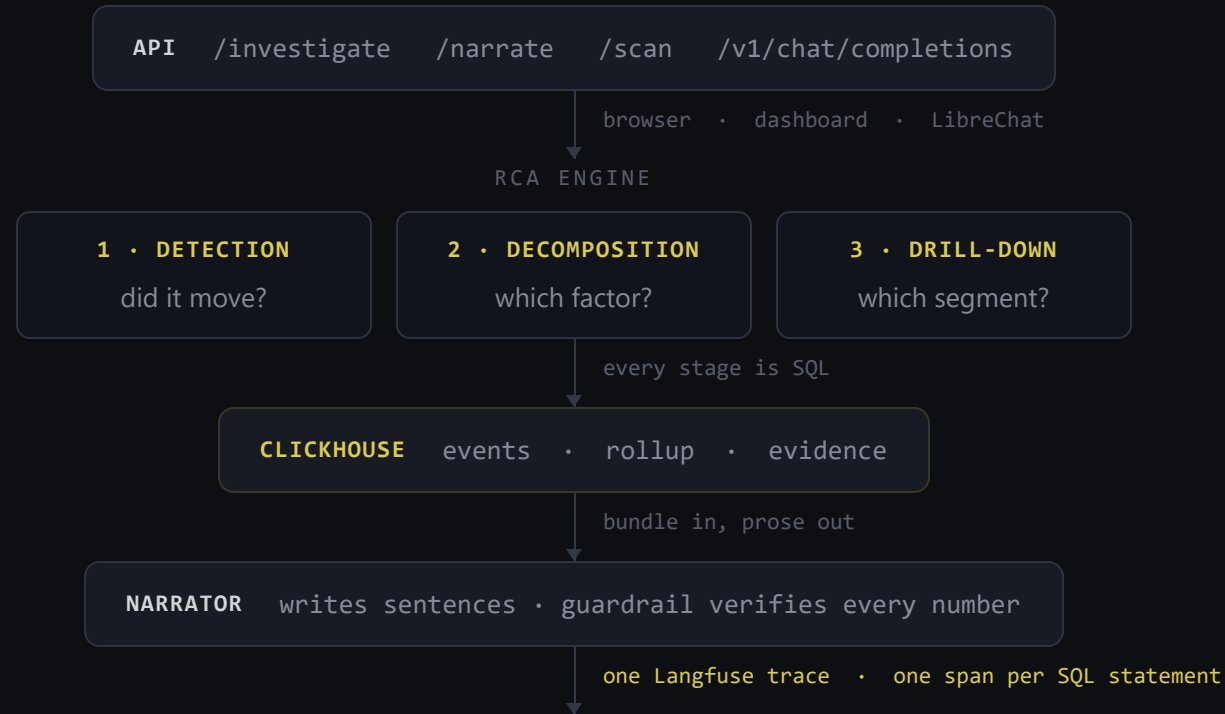
```
scanning the intersection
```

```
region AND os_version      fill rate roughly halved
```

A single-dimension scan is **structurally blind** to this class of anomaly — averaging over everyone else dilutes it below any sensible floor. The search descends through combinations for exactly this reason.



# ClickHouse is the detective. The LLM is the journalist.



Results are written back into ClickHouse — investigation history is itself queryable in SQL.

# Not a claim. A trace.

Every SQL statement is a span, nested under the investigation automatically. The span tree **is** the pipeline:

**investigation:fill\_rate**

- detect**
- decompose**
- drilldown**

Open any span and the ClickHouse query that produced the number is right there, with its result.

The LLM has no database access and performs no arithmetic.

The screenshot displays the Trace interface for an investigation titled "investigation:fill\_rate: 92b83659e32696e198f5303d413fbddf". The span tree on the left shows a hierarchy of operations: investigation:fill\_rate (8.39s) with an anomaly score of -128.58, investigation:fill\_rate (8.39s), detect (1.31s), scan:fill\_rate (0.24s), clickhouse-query (0.38s), sqlwindow-metric:fill\_rate (0.17s), sqlwindow-metric:fill\_rate (0.17s), sqlwindow-metric:fill\_rate (0.17s), sqlwindow-metric:fill\_rate (0.17s), decompose (0.20s), sqlfactor-sums (0.20s), drilldown (5.41s), sql:population:0 (0.20s), depth:0:os\_version (2.55s), sql:contribution:region (0.21s), sql:contribution:country (0.21s), sql:contribution:os\_version (0.21s), sql:contribution:device\_model (0.21s), and sql:contribution:ad\_format (0.21s).

The right panel shows the details for the "clickhouse-query" span, which occurred on 2026-08-02 at 14:32:29.227 with a latency of 0.38s and environment "default". The query is a SELECT statement aggregating data by hour. The output table shows the following data:

| Path      | Value                                           |
|-----------|-------------------------------------------------|
| row_count | 840                                             |
| columns   | ["hour", "requests", "fills", ...3 more]        |
| 0         | "hour"                                          |
| 1         | "requests"                                      |
| 2         | "fills"                                         |
| 3         | "impressions"                                   |
| 4         | "clicks"                                        |
| 5         | "revenue"                                       |
| sample    | [..., ..., ..., ..., ...]                       |
| > 0       | ["2026-06-01T00:00:00Z", 7368, 5801, ...3 more] |
| > 1       | ["2026-06-01T01:00:00Z", 7769, 6065, ...3 more] |

# Every dimension checked. Then it stops.

At each depth the engine scores **every value of every dimension** — region, country, OS, device, format, category, tier, vertical, campaign type, app, advertiser.

Then **depth-1:stop**: nothing left is both material and disproportionate, so the search ends rather than inventing depth.

The ruled-out list is not written by the model. It is what the search actually examined and cleared.

The screenshot displays a search engine interface with a query tree on the left and a SQL query result on the right.

**Query Tree (Left):**

- sqlfactor-sums (0.20s)
  - drilldown (5.41s)
    - sql:population:0 (0.20s)
      - depth-0:os\_version (2.55s)
        - sql:contribution:region (0.21s)
        - sql:contribution:country (0.21s)
        - sql:contribution:os\_version (0.21s)
        - sql:contribution:device\_model (0.21s)
        - sql:contribution:ad\_format (0.21s)
        - sql:contribution:category (0.21s)
        - sql:contribution:publisher\_tier (0.21s)
        - sql:contribution:vertical (0.22s)
        - sql:contribution:campaign\_type (0.21s)
        - sql:contribution:app\_id (0.40s)
        - sql:contribution:advertiser\_id (0.23s)
      - sql:population:1 (0.21s)
        - depth-1:stop (2.23s)
          - sql:contribution:region (0.21s)
          - sql:contribution:country (0.21s)
          - sql:contribution:device\_model (0.21s)

**SQL Query (Right):**

```
SELECT hour,
sum(requests) AS requests,
sum(fills) AS fills,
sum(impressions) AS impressions,
sum(clicks) AS clicks,
sum(revenue) AS revenue
FROM hourly_summary
GROUP BY hour
ORDER BY hour
```

**Output Table:**

| Path      | Value                                           |
|-----------|-------------------------------------------------|
| row_count | 840                                             |
| columns   | ["hour", "requests", "fills", ...3 more]        |
| 0         | "hour"                                          |
| 1         | "requests"                                      |
| 2         | "fills"                                         |
| 3         | "impressions"                                   |
| 4         | "clicks"                                        |
| 5         | "revenue"                                       |
| sample    | [... ..]                                        |
| > 0       | ["2026-06-01T00:00:00Z", 7368, 5881, ...3 more] |
| > 1       | ["2026-06-01T01:00:00Z", 7769, 6065, ...3 more] |
| > 2       | ["2026-06-01T02:00:00Z", 7966, 6240, ...3 more] |

## DETECTION

# Three detectors. Thresholds learned, not guessed.

| DETECTOR                | BASELINE                                                               | STRENGTH                               |
|-------------------------|------------------------------------------------------------------------|----------------------------------------|
| <b>robust_z</b>         | Same weekday and hour, trailing weeks — median centre, MAD spread      | Deterministic, fully traceable         |
| <b>seasonal_ml</b>      | Per weekday × hour profile built from all history, scored on residuals | Pools hundreds of points, far steadier |
| <b>isolation_forest</b> | scikit-learn over a per-hour feature vector                            | Catches unusual metric combinations    |

### Calibrated per metric

A ratio over millions of requests is stable — a small move is real. One over a few thousand clicks swings wildly — the same move is noise. Each metric's floor is measured from its own volatility, so one shared threshold cannot drown the system in false positives.

### Every factor, not just revenue

A regression inside one factor can be masked by growth in another, leaving the headline metric flat while something is genuinely broken underneath. Revenue-only detection never sees it.

# Every number reproducible. The model cannot invent one.

## The bundle carries its own receipts

Each investigation stores the anomaly, the factor split, the drill-down path, the ruled-out list — and `queries[]`, the SQL behind every figure. Any number can be recomputed independently.

## Deterministic by construction

`/investigate` has no LLM in the path. Call it twice, diff the result — same input, same bundle.

## The guardrail

Every number in the prose is extracted and checked against the evidence. Anything absent is rejected.

- ✓ a figure that appears in the bundle
- ✓ the same figure, sensibly rounded
- ✗ a figure that appears nowhere
- ✗ a real figure wearing invented units

That last one matters: a genuine number restated with an added currency symbol or scale suffix passes a naive digit check while overstating the value by orders of magnitude.

The verdict is stored on the bundle, so a failed check is **visible**, not silent.

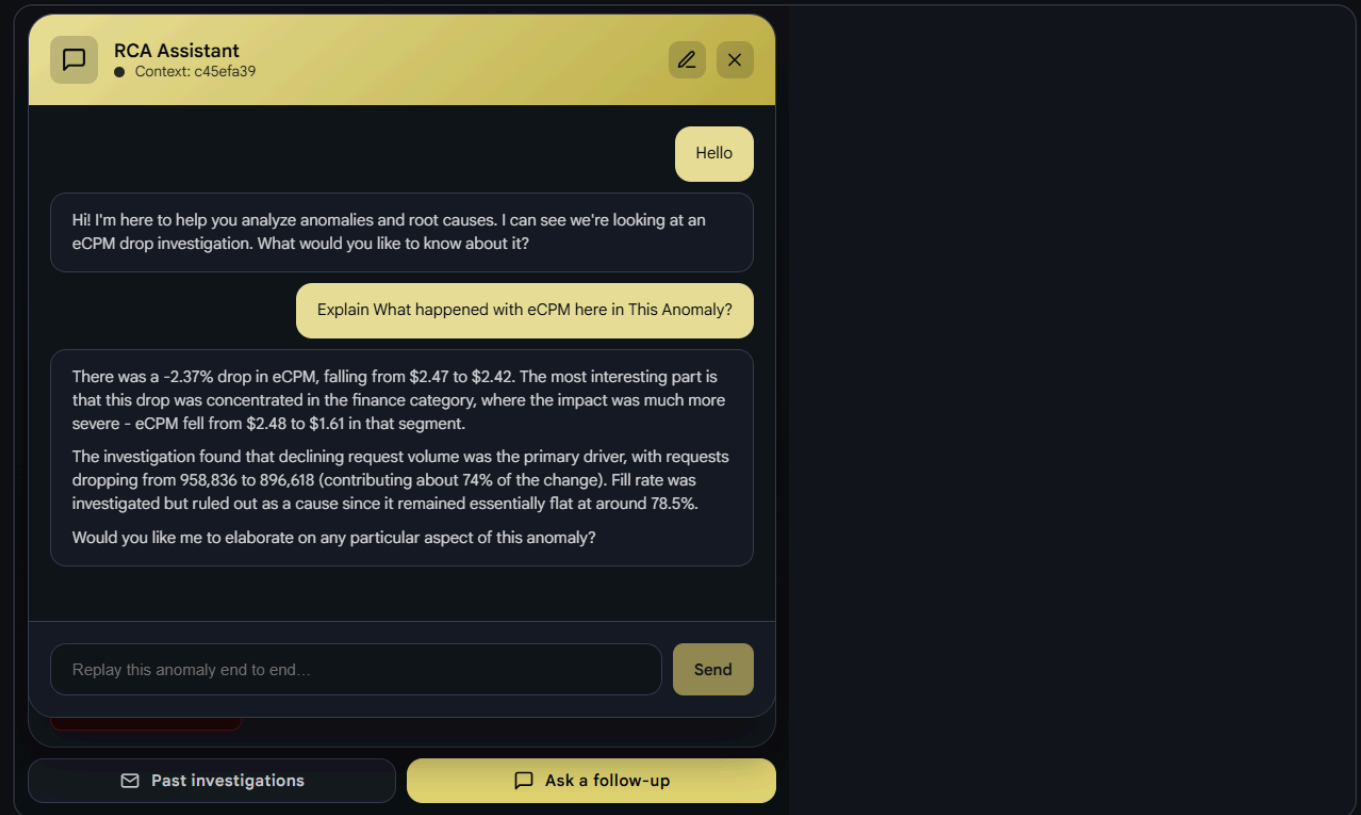
# Point it at a window it has never seen.

No case list, no ground truth, no hints. **POST /scan** sweeps a date range and finds the incidents itself.

- Every metric, **and every value of every dimension**
  - not just population aggregates
- Echoes of one underlying event folded together
- Findings ranked, localized, and each one a full Evidence Bundle

A large collapse confined to a small slice barely moves the global metric. Segment-level sweeping is what makes those findable at all.

Then ask follow-ups in plain English — the conversation carries the investigation with it.



# Runs today. Scales by construction.

## Cost is bounded, not linear

A targeted investigation is a handful of grouped scans over a rollup. A full blind sweep is roughly **50 queries**. Greedy search with a stop criterion keeps that flat as dimensions grow, instead of exploding combinatorially.

## Deployed properly

Docker Compose on EC2 behind nginx and TLS. Secrets in SSM Parameter Store, read through an IAM instance role — none in the repo, none in the image, no keys on the host. Reproducible from [deploy/](#).

## Where it fits

The engine is dimension-agnostic — it reads the dimension list from config and scores whatever it is given. Nothing in it is specific to ad tech.

---

[clickathon.kangasys.com](#) • live demo

[traces.kangasys.com](#) • Langfuse

[chat.kangasys.com](#) • LibreChat

[github.com/Rohanmrao/Clickathon2026](#)

Jalagaara Gang • Click-a-thon 2026